

partner whom she prefers to her partner in the stable matching M .

25.2-5

The *stable-roommates problem* is similar to the stable-marriage problem, except that the graph is a complete graph, not bipartite, with an even number of vertices. Each vertex represents a person, and each person ranks all the other people. The definitions of blocking pairs and stable matching extend in the natural way: a blocking pair comprises two people who both prefer each other to their current partner, and a matching is stable if there are no blocking pairs. For example, consider four people—Wendy, Xenia, Yolanda, and Zelda—with the following preference lists:

Wendy: Xenia, Yolanda, Zelda
Xenia: Wendy, Zelda, Yolanda
Yolanda: Wendy, Zelda, Xenia
Zelda: Xenia, Yolanda, Wendy

You can verify that the following matching is stable:

Wendy and Xenia
Yolanda and Zelda

Unlike the stable-marriage problem, the stable-roommates problem can have inputs for which no stable matching exists. Find such an input and explain why no stable matching exists.

25.3 The Hungarian algorithm for the assignment problem

Let us once again add some information to a complete bipartite graph $G = (V, E)$, where $V = L \cup R$. This time, instead of having the vertices of each side rank the vertices on the other side, we assign a weight to each edge. Again, let's assume that the vertex sets L and R each contain n vertices, so that the graph contains n^2 edges. For $l \in L$ and $r \in R$, denote the weight of edge (l, r) by $w(l, r)$, which represents the utility gained by matching vertex l with vertex r .

The goal is to find a perfect matching M^* (see Exercises 25.1-5 and 25.1-6) whose edges have the maximum total weight over all perfect matchings. That is, letting $w(M) = \sum_{(l,r) \in M} w(l, r)$ denote the total weight of the edges in matching M , we want to find a perfect matching M^* such that

$$w(M^*) = \max \{w(M) : M \text{ is a perfect matching}\}.$$

We call finding such a maximum-weight perfect matching the **assignment problem**. A solution to the assignment problem is a perfect matching that maximizes the total utility. Like the stable-marriage problem, the assignment problem finds a matching that is “good,” but with a different definition of good: maximizing total value rather than achieving stability.

Although you could enumerate all $n!$ perfect matchings to solve the assignment problem, an algorithm known as the **Hungarian algorithm** solves it much faster. This section will prove an $O(n^4)$ time bound, and Problem 25-2 asks you to refine the algorithm to reduce the running time to $O(n^3)$. Instead of working with the complete bipartite graph G , the Hungarian algorithm works with a subgraph of G called the “equality subgraph.” The equality subgraph, which is defined below, changes over time and has the beneficial property that any perfect matching in the equality subgraph is also an optimal solution to the assignment problem.

The equality subgraph depends on assigning an attribute h to each vertex. We call h the **label** of a vertex, and we say that h is a **feasible vertex labeling** of G if

$$l.h + r.h \geq w(l, r) \text{ for all } l \in L \text{ and } r \in R.$$

A feasible vertex labeling always exists, such as the **default vertex labeling** given by

$$l.h = \max \{w(l, r) : r \in R\} \text{ for all } l \in L, \quad (25.1)$$

$$r.h = 0 \text{ for all } r \in R. \quad (25.2)$$

Given a feasible vertex labeling h , the *equality subgraph* $G_h = (V, E_h)$ of G consists of the same vertices as G and the subset of edges

$$E_h = \{(l, r) \in E : l.h + r.h = w(l, r)\}.$$

The following theorem ties together a perfect matching in an equality subgraph and an optimal solution to the assignment problem.

Theorem 25.14

Let $G = (V, E)$, where $V = L \cup R$, be a complete bipartite graph where each edge $(l, r) \in E$ has weight $w(l, r)$. Let h be a feasible vertex labeling of G and G_h be the equality subgraph of G . If G_h contains a perfect matching M^* , then M^* is an optimal solution to the assignment problem on G .

Proof If G_h contains a perfect matching M^* , then because G_h and G have the same sets of vertices, M^* is also a perfect matching in G . Because each edge of M^* belongs to G_h and each vertex has exactly one incident edge from any perfect matching, we have

$$\begin{aligned} w(M^*) &= \sum_{(l,r) \in M^*} w(l, r) \\ &= \sum_{(l,r) \in M^*} (l.h + r.h) \quad (\text{because all edges in } M^* \text{ belong to } G_h) \\ &= \sum_{l \in L} l.h + \sum_{r \in R} r.h \quad (\text{because } M^* \text{ is a perfect matching}). \end{aligned}$$

Letting M be any perfect matching in G , we have

$$\begin{aligned} w(M) &= \sum_{(l,r) \in M} w(l, r) \\ &\leq \sum_{(l,r) \in M} (l.h + r.h) \quad (\text{because } h \text{ is a feasible vertex labeling}) \\ &= \sum_{l \in L} l.h + \sum_{r \in R} r.h \quad (\text{because } M \text{ is a perfect matching}). \end{aligned}$$

Thus, we have

$$w(M) \leq \sum_{l \in L} l.h + \sum_{r \in R} r.h = w(M^*) , \quad (25.3)$$

so that M^* is a maximum-weight perfect matching in G . ■

The goal now becomes finding a perfect matching in an equality subgraph. Which equality subgraph? It does not matter! We have free rein to not only choose an equality subgraph, but to change which equality subgraph we choose as we go along. We just need to find *some* perfect matching in *some* equality subgraph.

To understand the equality subgraph better, consider again the proof of Theorem 25.14 and, in the second half, let M be any matching. The proof is still valid, in particular, inequality (25.3): the weight of any matching is always at most the sum of the vertex labels. If we choose any set of vertex labels that define an equality subgraph, then a maximum-cardinality matching in this equality subgraph has total value at most the sum of the vertex labels. If the set of vertex labels is the “right” one, then it will have total value equal to $w(M^*)$, and a maximum-cardinality matching in the equality subgraph is also a maximum-weight perfect matching. The Hungarian algorithm repeatedly modifies the matching and the vertex labels in order to achieve this goal.

The Hungarian algorithm starts with any feasible vertex labeling h and any matching M in the equality subgraph G_h . It repeatedly finds an M -augmenting path P in G_h and, using Lemma 25.1, updates the matching to be $M \oplus P$, thereby incrementing the size of the matching. As long as there is some equality subgraph that contains an M -augmenting path, the size of the matching can increase, until a perfect matching is achieved.

Four questions arise:

1. What initial feasible vertex labeling should the algorithm start with? Answer: the default vertex labeling given by equations (25.1) and (25.2).

2. What initial matching in G_h should the algorithm start with?
Short answer: any matching, even an empty matching, but a greedy maximal matching works well.
3. If an M -augmenting path exists in G_h , how to find it? Short answer: use a variant of breadth-first search similar to the second phase of the procedure used in the Hopcroft-Karp algorithm to find a maximal set of shortest M -augmenting paths.
4. What if the search for an M -augmenting path fails? Short answer: update the feasible vertex labeling to bring in at least one new edge.

We'll elaborate on the short answers using the example that starts in Figure 25.4. Here, $L = \{l_1, l_2, \dots, l_7\}$ and $R = \{r_1, r_2, \dots, r_7\}$. The edge weights appear in the matrix shown in part (a), where the weight $w(l_i, r_j)$ appears in row i and column j . The feasible vertex labels, given by the default vertex labeling, appear to the left of and above the matrix. Matrix entries in red indicate edges (l_i, r_j) for which $l_i.h + r_j.h = w(l_i, r_j)$, that is, edges in the equality subgraph G_h appearing in part (b) of the figure.

Greedy maximal bipartite matching

There are several ways to implement a greedy method to find a maximal bipartite matching. The procedure GREEDY-BIPARTITE-MATCHING shows one. Edges in Figure 25.4(b) highlighted in blue indicate the initial greedy maximal matching in G_h . Exercise 25.3-2 asks you to show that the GREEDY-BIPARTITE-MATCHING procedure returns a matching that is at least half the size of a maximum matching.

GREEDY-BIPARTITE-MATCHING (G)

```

1  $M = \emptyset$ 
2 for each vertex  $l \in L$ 
3   if  $l$  has an unmatched neighbor in  $R$ 
4     choose any such unmatched neighbor  $r \in R$ 
```

```

5       $M = M \cup \{(l, r)\}$ 
6  return  $M$ 

```

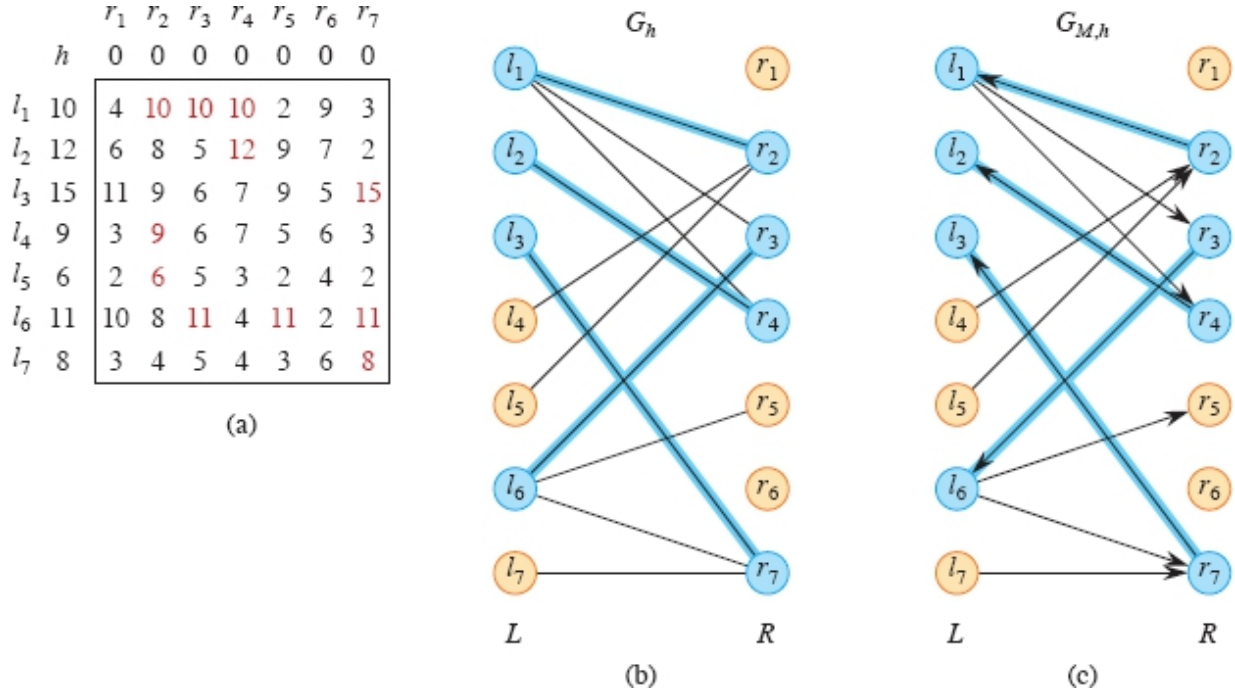


Figure 25.4 The start of the Hungarian algorithm. **(a)** The matrix of edge weights for a bipartite graph with $L = \{l_1, l_2, \dots, l_7\}$. The value in row i and column j indicates $w(l_i, r_j)$. Feasible vertex labels appear above and next to the matrix. Red entries correspond to edges in the equality subgraph. **(b)** The equality subgraph G_h . Edges highlighted in blue belong to the initial greedy maximal matching M . Blue vertices are matched, and tan vertices are unmatched. **(c)** The directed equality subgraph $G_{M,h}$ created from G_h by directing edges in M from R to L and all other edges from L to R .

Finding an M -augmenting path in G_h

To find an M -augmenting path in the equality subgraph G_h with a matching M , the Hungarian algorithm first creates the *directed equality subgraph* $G_{M,h}$ from G_h , just as the Hopcroft-Karp algorithm creates G_M from G . As in the Hopcroft-Karp algorithm, you can think of an M -augmenting path as starting from an unmatched vertex in L , ending

at an unmatched vertex in R , taking unmatched edges from L to R , and taking matched edges from R to L . Thus, $G_{M,h} = (V, E_{M,h})$, where

$$E_{M,h} = \{(l, r) : l \in L, r \in R, \text{ and } (l, r) \in E_h - M\} \text{ (edges from } L \text{ to } R) \\ \cup \{(r, l) : r \in R, l \in L, \text{ and } (l, r) \in M\} \text{ (edges from } R \text{ to } L).$$

Because an M -augmenting path in the directed equality subgraph $G_{M,h}$ is also an M -augmenting path in the equality subgraph G_h , it suffices to find M -augmenting paths in $G_{M,h}$. Figure 25.4(c) shows the directed equality subgraph $G_{M,h}$ corresponding to the equality subgraph G_h and matching M from part (b) of the figure.

With the directed equality subgraph $G_{M,h}$ in hand, the Hungarian algorithm searches for an M -augmenting path from any unmatched vertex in L to any unmatched vertex in R . Any exhaustive graph-search method suffices. Here, we'll use breadth-first search, starting from all the unmatched vertices in L (just as the Hopcroft-Karp algorithm does when creating the dag H), but stopping upon first discovering some unmatched vertex in R . Figure 25.5 shows the idea. To start from all the unmatched vertices in L , initialize the first-in, first-out queue with all the unmatched vertices in L , rather than just one source vertex. Unlike the dag H in the Hopcroft-Karp algorithm, here each vertex needs just one predecessor, so that the breadth-first search creates a *breadth-first forest* $F = (V_F, E_F)$. Each unmatched vertex in L is a root in F .

In Figure 25.5(g), the breadth-first search has found the M -augmenting path $\langle (l_4, r_2), (r_2, l_1), (l_1, r_3), (r_3, l_6), (l_6, r_5) \rangle$. Figure 25.6(a) shows the new matching created by taking the symmetric difference of the matching M in Figure 25.5(a) with this M -augmenting path.

When the search for an M -augmenting path fails

Having updated the matching M from an M -augmenting path, the Hungarian algorithm updates the directed equality subgraph $G_{M,h}$ according to the new matching and then starts a new breadth-first

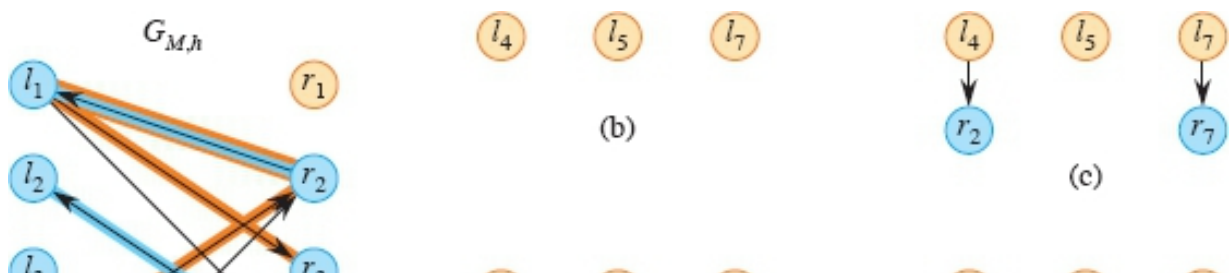
search from all the unmatched vertices in L . Figure 25.6 shows the start of this process, picking up from Figure 25.5.

In Figure 25.6(d), the queue contains vertices l_4 and l_3 . Neither of these vertices has an edge that leaves it, however, so that once these vertices are removed from the queue, the queue becomes empty. The search terminates at this point, before discovering an unmatched vertex in R to yield an M -augmenting path. Whenever this situation occurs, the most recently discovered vertices must belong to L . Why? Whenever an unmatched vertex in R is discovered, the search has found an M -augmenting path, and when a matched vertex in R is discovered, it has an unvisited neighbor in L , which the search can then discover.

Recall that we have the freedom to work with any equality subgraph. We can change the directed equality subgraph “on the fly,” as long we do not counteract the work already done. The Hungarian algorithm updates the feasible vertex labeling h to fulfill the following criteria:

1. No edge in the breadth-first forest F leaves the directed equality subgraph.
2. No edge in the matching M leaves the directed equality subgraph.
3. At least one edge (l, r) , where $l \in L \cap V_F$ and $r \in R - V_F$ goes into E_h , and hence into $E_{M,h}$. Therefore, at least one vertex in R will be newly discovered.

Thus, at least one new edge enters the directed equality subgraph, and any edge that leaves the directed equality subgraph belongs to neither the matching M nor the breadth-first forest F . Newly discovered vertices in R are enqueued, but their distances are not necessarily 1 greater than the distances of the most recently discovered vertices in L .



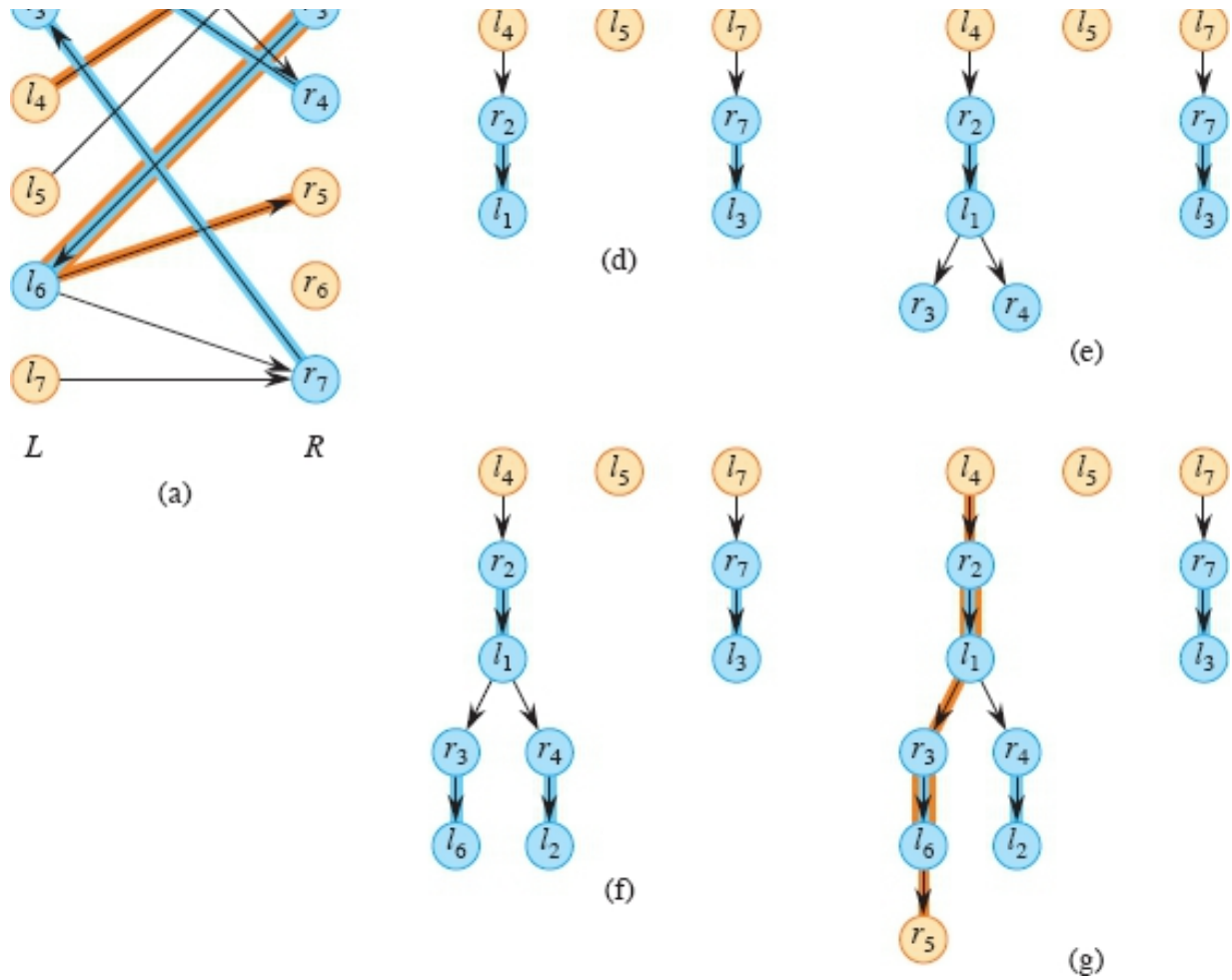


Figure 25.5 Finding an M -augmenting path in $G_{M,h}$ by breadth-first search. (a) The directed equality subgraph $G_{M,h}$ from Figure 25.4(c). (b)–(g) Successive versions of the breadth-first forest F , shown as the vertices at each distance from the roots—the unmatched vertices in L —are discovered. In parts (b)–(f), the layer of vertices closest to the bottom of the figure are those in the first-in, first-out queue. For example, in part (b), the queue contains the roots $\langle l_4, l_5, l_7 \rangle$, and in part (e), the queue contains $\langle r_3, r_4 \rangle$, at distance 3 from the roots. In part (g), the unmatched vertex r_5 is discovered, so the breadth-first search terminates. The path $\langle (l_4, r_2), (r_2, l_1), (l_1, r_3), (r_3, l_6), (l_6, r_5) \rangle$, highlighted in orange in parts (a) and (g), is an M -augmenting path. Taking its symmetric difference with the matching M yields a new matching with one more edge than M .

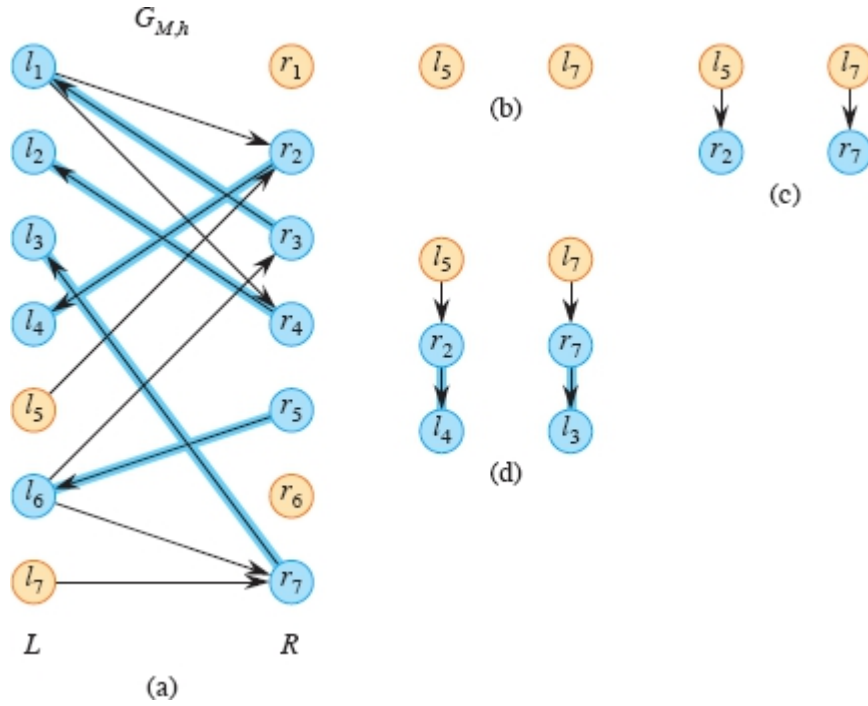


Figure 25.6 (a) The new matching M and the new directed equality subgraph $G_{M,h}$ after updating the matching in Figure 25.5(a) with the M -augmenting path in Figure 25.5(g). (b)–(d) Successive versions of the breadth-first forest F in a new breadth-first search with roots l_5 and l_7 . After the vertices l_4 and l_3 in part (d) have been removed from the queue, the queue becomes empty before the search can discover an unmatched vertex in R .

To update the feasible vertex labeling, the Hungarian algorithm first computes the value

$$\delta = \min \{l.h + r.h - w(l, r) : l \in F_L \text{ and } r \in R - F_R\} , \quad (25.4)$$

where $F_L = L \cap V_F$ and $F_R = R \cap V_F$ denote the vertices in the breadth-first forest F that belong to L and R , respectively. That is, δ is the smallest difference by which an edge incident on a vertex in F_L missed being in the current equality subgraph G_h . The Hungarian algorithm then creates a new feasible vertex labeling, say h' , by subtracting δ from $l.h$ for all vertices $l \in F_L$ and adding δ to $r.h$ for all vertices $r \in F_R$:

$$v.h' = \begin{cases} v.h - \delta & \text{if } v \in F_L, \\ v.h + \delta & \text{if } v \in F_R, \\ v.h & \text{otherwise } (v \in V - V_F). \end{cases} \quad (25.5)$$

The following lemma shows that these changes achieve the three criteria above.

Lemma 25.15

Let h be a feasible vertex labeling for the complete bipartite graph G with equality subgraph G_h , and let M be a matching for G_h and F be a breadth-first forest being constructed for the directed equality subgraph $G_{M,h}$. Then, the labeling h' in equation (25.5) is a feasible vertex labeling for G with the following properties:

1. If (u, v) is an edge in the breadth-first forest F for $G_{M,h}$, then $(u, v) \in E_{M,h'}$.
2. If (l, r) belongs to the matching M for G_h , then $(r, l) \in E_{M,h'}$.
3. There exist vertices $l \in F_L$ and $r \in R - F_R$ such that $(l, r) \notin E_{M,h}$ but $(l, r) \in E_{M,h'}$.

Proof We first show that h' is a feasible vertex labeling for G . Because h is a feasible vertex labeling, we have $l.h + r.h \geq w(l, r)$ for all $l \in L$ and $r \in R$. In order for h' to not be a feasible vertex labeling, we would need $l.h' + r.h' < l.h + r.h$ for some $l \in L$ and $r \in R$. The only way this could occur would be for some $l \in F_L$ and $r \in R - F_R$. In this instance, the amount of the decrease equals δ , so that $l.h' + r.h' = l.h - \delta + r.h$. By equation (25.4), we have that $l.h - \delta + r.h \geq w(l, r)$ for any $l \in F_L$ and $r \in R - F_R$, so that $l.h' + r.h' \geq w(l, r)$. For all other edges, we have $l.h' + r.h' \geq l.h + r.h \geq w(l, r)$. Thus, h' is a feasible vertex labeling.

Now we show that each of the three desired properties holds:

1. If $l \in F_L$ and $r \in F_R$, then we have $l.h' + r.h' = l.h + r.h$ because δ is added to the label of l and subtracted from the label of r .

Therefore, if an edge belongs to F for the directed graph $G_{M,h}$, it also belongs to $G_{M,h'}$.

2. We claim that at the time the Hungarian algorithm computes the new feasible vertex labeling h' , for every edge $(l, r) \in M$, we have $l \in F_L$ if and only if $r \in F_R$. To see why, consider a matched vertex r and let $(l, r) \in M$. First suppose that $r \in F_R$, so that the search discovered r and enqueued it. When r was removed from the queue, l was discovered, so $l \in F_L$. Now suppose that $r \notin F_R$, so r is undiscovered. We will show that $l \notin F_L$. The only edge in $G_{M,h}$ that enters l is (r, l) , and since r is undiscovered, the search has not taken this edge; if $l \in F_L$, it is not because of the edge (r, l) . The only other way that a vertex in L can be in F_L is if it is a root of the search, but only unmatched vertices in L are roots and l is matched. Thus, $l \notin F_L$, and the claim is proved.

We already saw that $l \in F_L$ and $r \in F_R$ implies $l.h' + r.h' = l.h + r.h$. For the opposite case, when $l \in L - F_L$ and $r \in R - F_R$, we have that $l.h' = l.h$ and $r.h' = r.h$, so that again $l.h' + r.h' = l.h + r.h$. Thus, if edge (l, r) is in the matching M for the equality graph G_h , then $(r, l) \in E_{M,h'}$.

3. Let (l, r) be an edge not in E_h such that $l \in F_L$, $r \in R - F_R$, and $\delta = l.h + r.h - w(l, r)$. By the definition of δ , there is at least one such edge. Then, we have

$$\begin{aligned} l.h' + r.h' &= l.h - \delta + r.h \\ &= l.h - (l.h + r.h - w(l, r)) + r.h \\ &= w(l, r), \end{aligned}$$

and thus $(l, r) \in E_{h'}$. Since (l, r) is not in E_h , it is not in the matching M , so that in $E_{M,h'}$ it must be directed from L to R . Thus, $(l, r) \in E_{M,h'}$.

■

It is possible for an edge to belong to $E_{M,h}$ but not to $E_{M,h'}$. By Lemma 25.15, any such edge belongs neither to the matching M nor to the breadth-first forest F at the time that the new feasible vertex labeling h' is computed. (See Exercise 25.3-3.)

Going back to Figure 25.6(d), the queue became empty before an M -augmenting path was found. Figure 25.7 shows the next steps taken by the algorithm. The value of $\delta = 1$ is achieved by the edge (l_5, r_3) because in Figure 25.4(a), $l_5.h + r_3.h - w(l_5, r_3) = 6 + 0 - 5 = 1$. In Figure 25.7(a), the values of $l_3.h$, $l_4.h$, $l_5.h$, and $l_7.h$ have decreased by 1 and the values of $r_2.h$ and $r_7.h$ have increased by 1 because these vertices are in F . As a result, the edges (l_1, r_2) and (l_6, r_7) leave $G_{M,h}$ and the edge (l_5, r_3) enters. Figure 25.7(b) shows the new directed equality subgraph $G_{M,h}$. With edge (l_5, r_3) now in $G_{M,h}$, Figure 25.7(c) shows that this edge is added to the breadth-first forest F , and r_3 is added to the queue. Parts (c)–(f) show the breadth-first forest continuing to be built until in part (f), the queue once again becomes empty after vertex l_2 , which has no edges leaving, is removed. Again, the algorithm must update the feasible vertex labeling and the directed equality subgraph. Now the value of $\delta = 1$ is achieved by three edges: (l_1, r_6) , (l_5, r_6) , and (l_7, r_6) .

As Figure 25.8 shows in parts (a) and (b), these edges enter $G_{M,h}$, and edge (l_6, r_3) leaves. Part (c) shows that edge (l_1, r_6) is added to the breadth-first forest. (Either of edges (l_5, r_6) or (l_7, r_6) could have been added instead.) Because r_6 is unmatched, the search has found the M -augmenting path $\langle (l_5, r_3), (r_3, l_1), (l_1, r_6) \rangle$, highlighted in orange.

Figure 25.9(a) shows $G_{M,h}$ after the matching M has been updated by taking its symmetric difference with the M -augmenting path. The Hungarian algorithm starts its last breadth-first search, with vertex l_7 as the only root. The search proceeds as shown in parts (b)–(h) of the figure, until the queue becomes empty after removing l_4 . This time, we find that $\delta = 2$, achieved by the five edges (l_2, r_5) , (l_3, r_1) , (l_4, r_5) , $(l_5,$

r_1), and (l_5, r_5) , each of which enters $G_{M,h}$. Figure 25.10(a) shows the results of decreasing the feasible vertex label of each vertex in F_L by 2 and increasing the feasible vertex label of each vertex in F_R by 2, and Figure 25.10(b) shows the resulting directed equality subgraph $G_{M,h}$. Part (c) shows that edge (l_3, r_1) is added to the breadth-first forest. Since r_1 is an unmatched vertex, the search terminates, having found the M -augmenting path $\langle (l_7, r_7), (r_7, l_3), (l_3, r_1) \rangle$, highlighted in orange. If r_1 had been matched, vertex r_5 would also have been added to the breadth-first forest, with any of l_2 , l_4 , or l_5 as its parent.

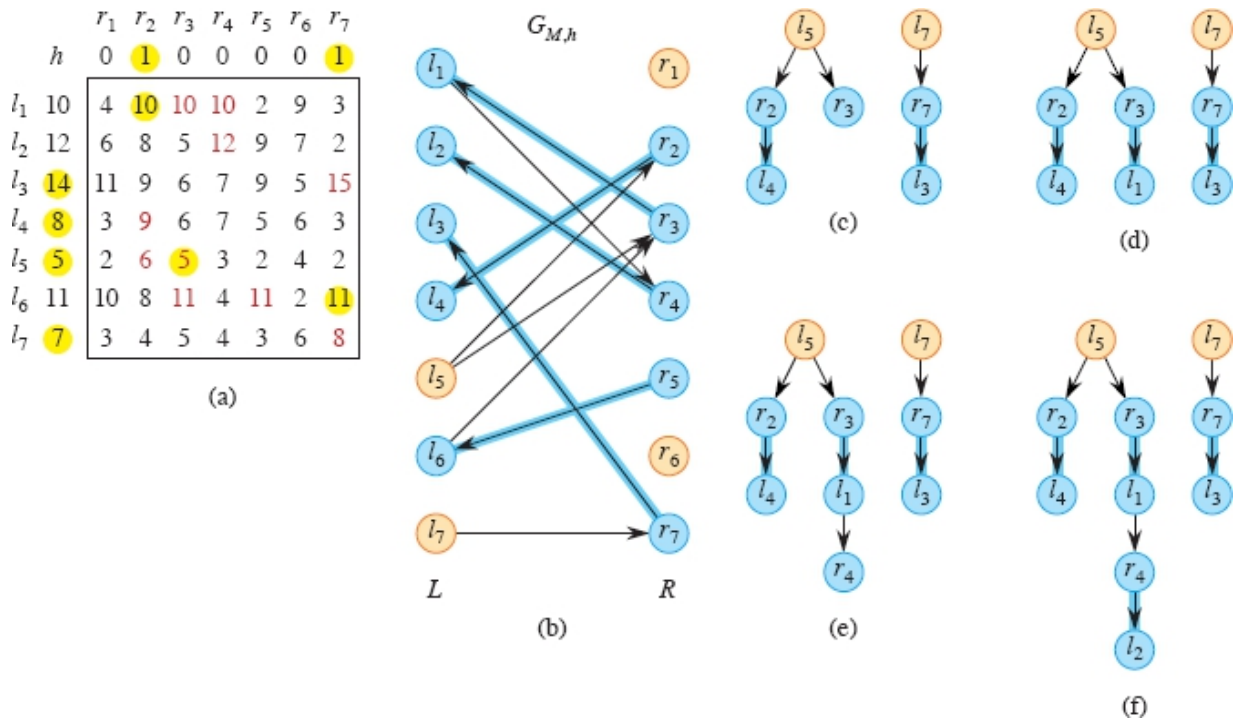


Figure 25.7 Updating the feasible vertex labeling and the directed equality subgraph $G_{M,h}$ when the queue becomes empty before finding an M -augmenting path. (a) With $\delta = 1$, the values of $l_3.h$, $l_4.h$, $l_5.h$, and $l_7.h$ decreased by 1 and $r_2.h$ and $r_7.h$ increased by 1. Edges (l_1, r_2) and (l_6, r_7) leave $G_{M,h}$, and edge (l_5, r_3) enters. These changes are highlighted in yellow. (b) The resulting directed equality subgraph $G_{M,h}$. (c)–(f) With edge (l_5, r_3) added to the breadth-first forest and r_3 added to the queue, the breadth-first search continues until the queue once again becomes empty in part (f).

After updating the matching M , the algorithm arrives at the perfect matching shown for the equality subgraph G_h in Figure 25.11. By Theorem 25.14, the edges in M form an optimal solution to the original assignment problem given in the matrix. Here, the weights of edges (l_1, r_6) , (l_2, r_4) , (l_3, r_1) , (l_4, r_2) , (l_5, r_3) , (l_6, r_5) , and (l_7, r_7) sum to 65, which is the maximum weight of any matching.

The weight of the maximum-weight matching equals the sum of all the feasible vertex labels. These problems—maximizing the weight of a matching and minimizing the sum of the feasible vertex labels—are “duals” of each other, in a similar vein to how the value of a maximum flow equals the capacity of a minimum cut. Section 29.3 explores duality in more depth.

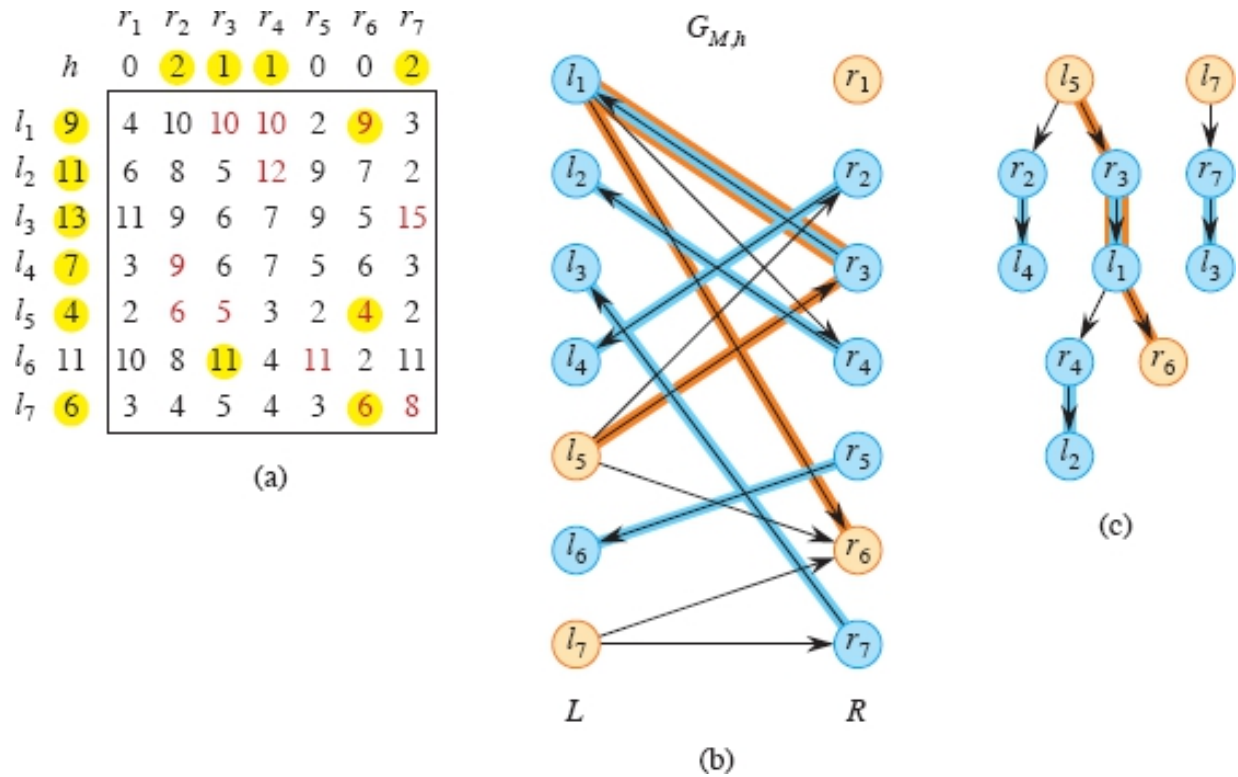


Figure 25.8 Another update to the feasible vertex labeling and directed equality subgraph $G_{M,h}$ because the queue became empty before finding an M -augmenting path. (a) With $\delta = 1$, the values of $l_1.h$, $l_2.h$, $l_3.h$, $l_4.h$, $l_5.h$, and $l_7.h$ decrease by 1, and $r_2.h$, $r_3.h$, $r_4.h$, and $r_7.h$ increase by 1. Edge (l_6, r_3) leaves $G_{M,h}$, and edges (l_1, r_6) , (l_5, r_6) and (l_7, r_6) enter. (b) The resulting directed equality subgraph $G_{M,h}$. (c) With edge (l_1, r_6) added to the breadth-first forest and r_6 unmatched, the search terminates, having found the M -augmenting path $\langle (l_5, r_3), (r_3, l_1), (l_1, r_6) \rangle$, highlighted in orange in parts (b) and (c).

The Hungarian algorithm

The procedure HUNGARIAN on page 737 and its subroutine FIND-AUGMENTING-PATH on page 738 follow the steps we have just seen. The third property in Lemma 25.15 ensures that in line 23 of FIND-AUGMENTING-PATH the queue Q is nonempty. The pseudocode uses the attribute π to indicate predecessor vertices in the breadth-first forest. Instead of coloring vertices, as in the BFS procedure on page 556, the search puts the discovered vertices into the sets F_L and F_R .

Because the Hungarian algorithm does not need breadth-first distances, the pseudocode omits the d attribute computed by the BFS procedure.

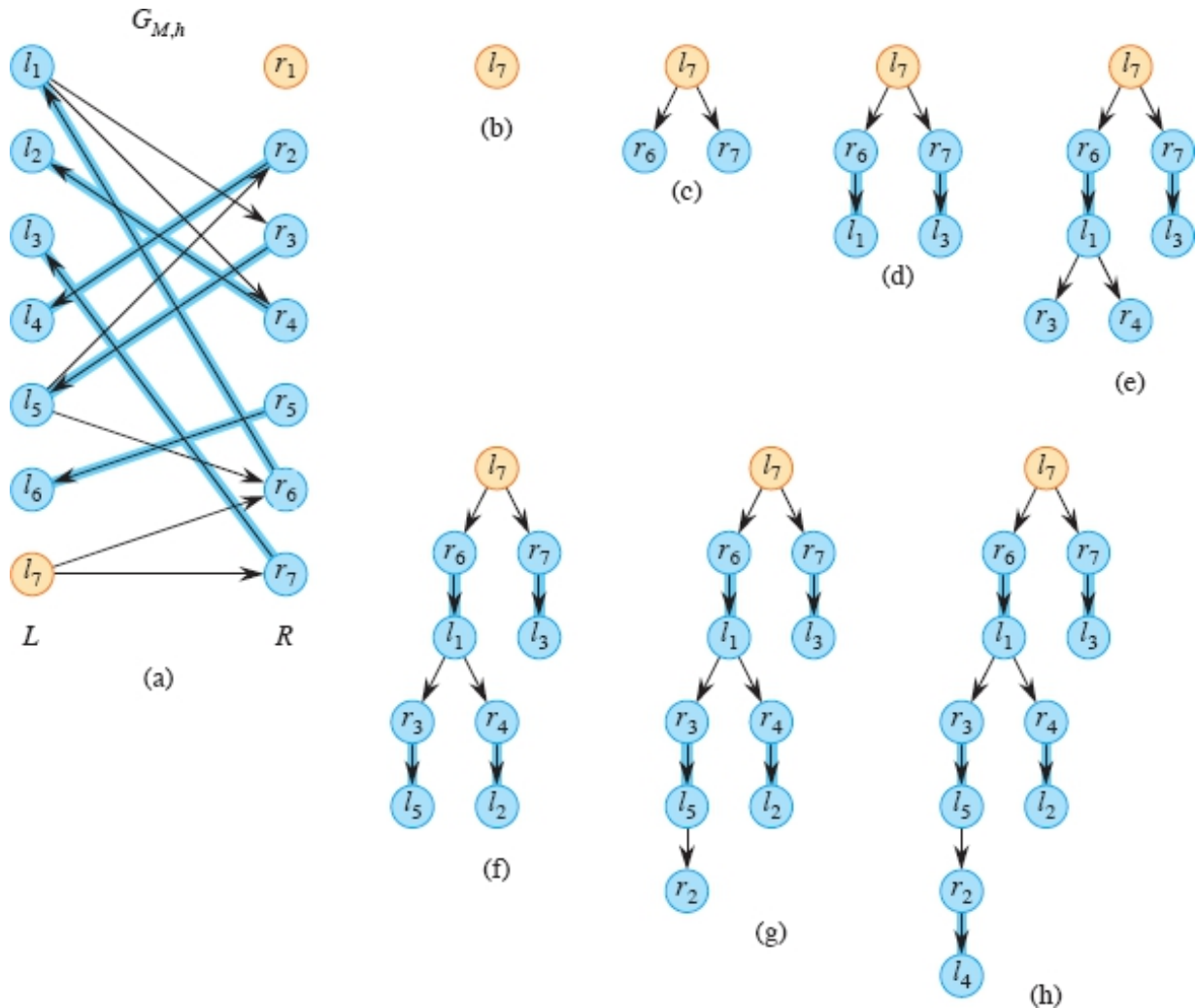


Figure 25.9 (a) The new matching M and the new directed equality subgraph $G_{M,h}$ after updating the matching in Figure 25.8 with the M -augmenting path in Figure 25.8 parts (b) and (c). (b)–(h) Successive versions of the breadth-first forest F in a new breadth-first search with root l_7 . After the vertex l_4 in part (h) has been removed from the queue, the queue becomes empty before the search discovers an unmatched vertex in R .

Now, let's see why the Hungarian algorithm runs in $O(n^4)$ time, where $|V| = n/2$ and $|E| = n^2$ in the original graph G . (Below we outline how to reduce the running time to $O(n^3)$.) You can go through the pseudocode of HUNGARIAN to verify that lines 1–6 and 11 take

$O(n^2)$ time. The **while** loop of lines 7–10 iterates at most n times, since each iteration increases the size of the matching M by 1. Each test in line 7 can take constant time by just checking whether $|M| < n$, each update of M in line 9 takes $O(n)$ time, and the updates in line 10 take $O(n^2)$ time.

To achieve the $O(n^4)$ time bound, it remains to show that each call of FIND-AUGMENTING-PATH runs in $O(n^3)$ time. Let's call each execution of lines 10–22 a *growth step*. Ignoring the growth steps, you can verify that FIND-AUGMENTING-PATH is a breadth-first search. With the sets F_L and F_R represented appropriately, the breadth-first search takes $O(V + E) = O(n^2)$ time. Within a call of FIND-AUGMENTING-PATH, at most n growth steps can occur, since each growth step is guaranteed to discover at least one vertex in R . Since there are at most n^2 edges in $G_{M,h}$, the **for** loop of lines 16–22 iterates at most n^2 times per call of FIND-AUGMENTING-PATH. The bottleneck is lines 10 and 15, which take $O(n^2)$ time, so that FIND-AUGMENTING-PATH takes $O(n^3)$ time.

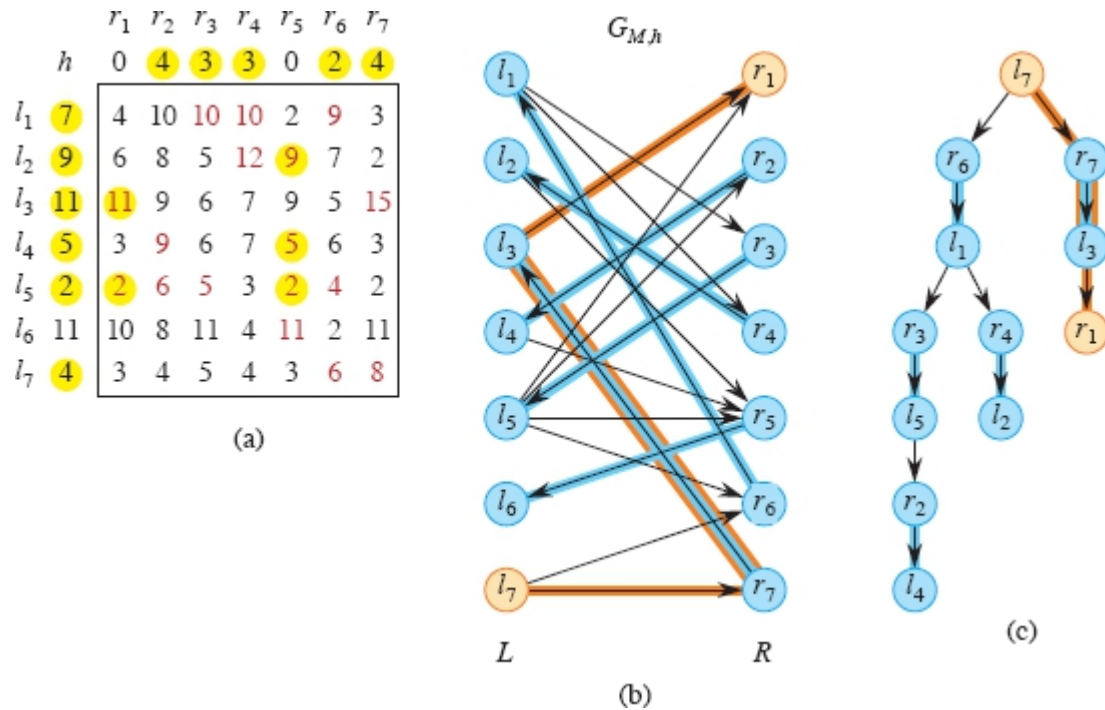


Figure 25.10 Updating the feasible vertex labeling and directed equality subgraph $G_{M,h}$. (a) Here, $\delta = 2$, so the values of $l_1.h$, $l_2.h$, $l_3.h$, $l_4.h$, $l_5.h$, and $l_7.h$ decreased by 2, and the values of $r_2.h$, $r_3.h$, $r_4.h$, $r_6.h$, and $r_7.h$ increased by 2. Edges (l_2, r_5) , (l_3, r_1) , (l_4, r_5) , (l_5, r_1) , and (l_5, r_5) enter $G_{M,h}$. (b) The resulting directed graph $G_{M,h}$. (c) With edge (l_3, r_1) added to the breadth-first forest and r_1 unmatched, the search terminates, having found the M -augmenting path $\langle (l_7, r_7), (r_7, l_3), (l_3, r_1) \rangle$, highlighted in orange in parts (b) and (c).

Exercise 25.3-5 asks you to show that reconstructing the directed equality subgraph $G_{M,h}$ in line 15 is actually unnecessary, so that its cost can be eliminated. Reducing the cost of computing δ in line 10 to $O(n)$ takes a little more effort and is the subject of Problem 25-2. With these changes, each call of FIND-AUGMENTING-PATH takes $O(n^2)$ time, so that the Hungarian algorithm runs in $O(n^3)$ time.

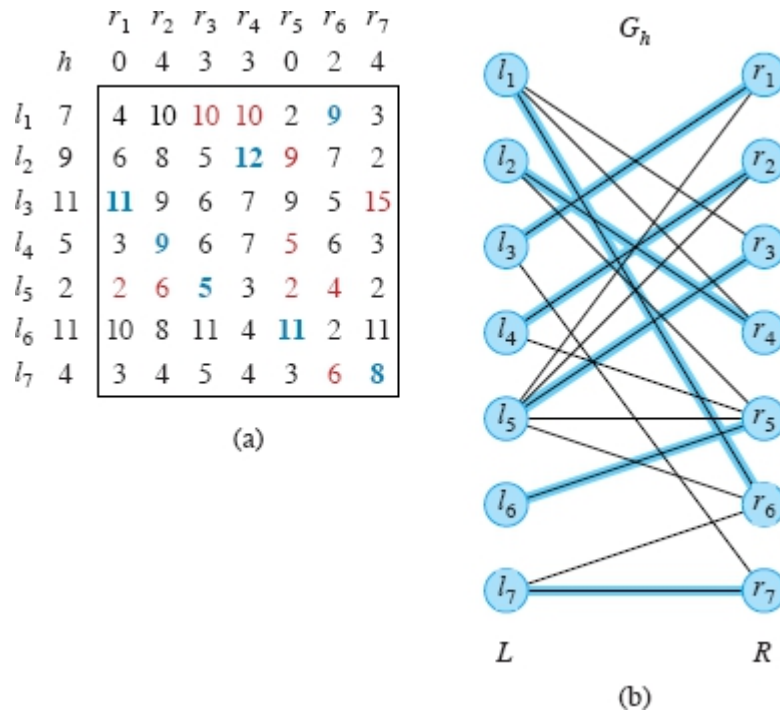


Figure 25.11 The final matching, shown for the equality subgraph G_h with blue edges and blue entries in the matrix. The weights of the edges in the matching sum to 65, which is the maximum for any matching in the original complete bipartite graph G , as well as the sum of all the final feasible vertex labels.

HUNGARIAN (G)

```

1 for each vertex  $l \in L$ 
2    $l.h = \max \{w(l, r) : r \in R\}$  // from equation (25.1)
3 for each vertex  $r \in R$ 
4    $r.h = 0$  // from equation (25.2)
5 let  $M$  be any matching in  $G_h$  (such as the matching returned by
   GREEDY-BIPARTITE-MATCHING)
6 from  $G$ ,  $M$ , and  $h$ , form the equality subgraph  $G_h$ 
   and the directed equality subgraph  $G_{M,h}$ 
7 while  $M$  is not a perfect matching in  $G_h$ 
8    $P = \text{FIND-AUGMENTING-PATH}(G_{M,h})$ 
9    $M = M \oplus P$ 

```

```

10    update the equality subgraph  $G_h$ 
        and the directed equality subgraph  $G_{M,h}$ 
11 return  $M$ 

```

FIND-AUGMENTING-PATH ($G_{M,h}$)

```

1  $Q = \emptyset$ 
2  $F_L = \emptyset$ 
3  $F_R = \emptyset$ 
4 for each unmatched vertex  $l \in L$ 
5      $l.\pi = \text{NIL}$ 
6     ENQUEUE ( $Q, l$ )
7      $F_L = F_L \cup \{l\}$            // forest  $F$  starts with unmatched
                                   // vertices in  $L$ 

8 repeat
9     if  $Q$  is empty                // ran out of vertices to search from?
10          $\delta = \min \{l.h + r.h - w(l, r) : l \in F_L \text{ and } r \in R - F_R\}$ 
11         for each vertex  $l \in F_L$ 
12              $l.h = l.h - \delta$            // relabel according to equation
                                           // (25.5)
13         for each vertex  $r \in F_R$ 
14              $r.h = r.h + \delta$            // relabel according to equation
                                           // (25.5)
15         from  $G, M$ , and  $h$ , form a new directed equality graph  $G_{M,h}$ 
16         for each new edge  $(l, r)$  in  $G_{M,h}$            // continue search with
                                                         // new edges
17             if  $r \notin F_R$ 
18                  $r.\pi = l$                  // discover  $r$ , add it to
                                                         // F
19                 if  $r$  is unmatched
20                     an  $M$ -augmenting path has been found
20                     (exit the repeat loop)
21                 else ENQUEUE           // can search from  $r$ 

```

	(Q, r)	later
22	$F_R = F_R \cup \{r\}$	
23	$u = \text{DEQUEUE}(Q)$	// search from u
24	for each neighbor v of u in $G_{M,h}$	
25	if $v \in L$	
26	$v.\pi = u$	
27	$F_L = F_L \cup \{v\}$	// discover v , add it to F
28	$\text{ENQUEUE}(Q, v)$	// can search from v later
29	elseif $v \notin F_R$	// $v \in R$, do same as lines 18–22
30	$v.\pi = u$	
31	if v is unmatched	
32	an M -augmenting path has been found	
	(exit the repeat loop)	
33	else $\text{ENQUEUE}(Q, v)$	
34	$F_R = F_R \cup \{v\}$	
35	until an M -augmenting path has been found	
36	using the predecessor attributes π , construct an M -augmenting path P by tracing back from the unmatched vertex in R	
37	return P	

Exercises

25.3-1

The FIND-AUGMENTING-PATH procedure checks in two places (lines 19 and 31) whether a vertex it discovers in R is unmatched. Show how to rewrite the pseudocode so that it checks for an unmatched vertex in R in only one place. What is the downside of doing so?

25.3-2

Show that for any bipartite graph, the GREEDY-BIPARTITE-MATCHING procedure on page 726 returns a matching at least half the size of a maximum matching.

25.3-3

Show that if an edge (l, r) belongs to the directed equality subgraph $G_{M,h}$ but is not a member of $G_{M,h'}$, where h' is given by equation (25.5), then $l \in L - F_L$ and $r \in F_R$ at the time that h' is computed.

25.3-4

At line 29 in the FIND-AUGMENTING-PATH procedure, it has already been established that $v \in R$. This line checks to see whether v is already discovered by testing whether $v \in F_R$. Why doesn't the procedure need to check whether v is already discovered for the case when $v \in L$, in lines 26–28?

25.3-5

Professor Hrabosky asserts that the directed equality subgraph $G_{M,h}$ must be constructed and maintained by the Hungarian algorithm, so that line 6 of HUNGARIAN and line 15 of FIND-AUGMENTING-PATH are required. Argue that the professor is incorrect by showing how to determine whether an edge belongs to $E_{M,h}$ without explicitly constructing $G_{M,h}$.

25.3-6

How can you modify the Hungarian algorithm to find a matching of vertices in L to vertices in R that minimizes, rather than maximizes, the sum of the edge weights in the matching?

25.3-7

How can an assignment problem with $|L| \neq |R|$ be modified so that the Hungarian algorithm solves it?

Problems

25-1 *Perfect matchings in a regular bipartite graph*

- a. Problem 20-3 asked about Euler tours in directed graphs. Prove that a connected, *undirected* graph $G = (V, E)$ has an Euler tour—a cycle